

Day 5

React Component Management



Introduction

When you create a component, React creates a virtual representation of that component in memory. This virtual representation is called the "**Virtual DOM**".

When the component's **state** or **props** change, ReactJS updates the Virtual DOM accordingly.

ReactJS applies the necessary patches to the actual DOM to reflect the changes. This process of updating the Virtual DOM and then applying the patches to the actual DOM is called "**Reconciliation**".

Class Component Life Management

In React, a **class component** has a well-defined lifecycle, which refers to the series of methods that are automatically called during creation, updating, and deletion.

These methods give you control over how a component behaves at different points in its lifecycle, such as when it's **first added** to the DOM, when it's **rendered** with new data, or when it's about to be removed.

The lifecycle of a class component can be categorized into **three phases**:

- **Mounting:** When a component is first created and added to the DOM.

Key Lifecycle Methods

Components

1. constructor:

- Initializes the component's state and methods.
- Called **once** before the component is rendered.

2. ComponentDidMount:

- Executed once **after** the component is mounted into the DOM.
- Useful for side effects like data fetching, subscriptions, or setting timers.

3. shouldComponentUpdate:

4. `componentDidUpdate`:

- Called after a component has been updated.
- Ideal for responding to prop or state changes, like re-fetching data.

5. `componentWillUnmount`:

- Executed right before a component is removed from the DOM.
- Useful for cleaning up resources like event listeners or canceling network requests.

Functional Components Management

Functional components are managed using **React Hooks** for state management, context, and other React features.

React Hooks

React Hooks are functions that allow you to use state and other React features in functional components.

They are called "**hooks**" because they allow you to "**hook into**" React state and lifecycle from functional components.

The important hooks for managing form components are:

- **useState**
- **useEffect**
- **useContext**
- **useReducer**
- **useCallback**

useState

The useState hook allows you to **add** state to a functional component.

It returns an array with two elements:

1. The **current state** value
2. A **function to update** that state.

Syntax:



```
1 const [state, setState] = useState
```


Example:



```
1 import React, { useState } from 'react'
2
3 function Counter() {
4
5   const [count, setCount] = useState(0)
6
7   const increment = () => {
8     setCount(count + 1);
9   };
10
11   return (
12     <div>
13       <p>Count: {count}</p>
14       <button onClick={increment}>Increment</button>
15     </div>
16   );
}
```

useEffect

The useEffect hook allows you to perform effects in function components, like subscriptions, or manually changing

It takes a function as an argument, v after every render.

It runs after every render, but you can specifying dependencies.

Syntax:



first argument

```
1 useEffect(() => {
```

```
2    // Code to run on every
```

Example:

```
1 import React, { useState, useEffect } from 'react'
2
3 function Timer() {
4   const [time, setTime] = useState(0);
5
6   useEffect(() => {
7     const intervalId = setInterval(() => {
8       setTime((prevTime) => prevTime + 1);
9     }, 1000);
10
11     return () => clearInterval(intervalId);
12   }, []);
13
14   return <div>Time: {time} seconds</div>;
15 }
```

cleanup function

dependency array

- The **useEffect** hook sets up a timer when the component mounts.

useContext

The **useContext** hook lets you consume **context** directly in a functional component, avoiding the need for nested **Consumer** components.

Context is a way to share data between components without having to pass it manually. You can think of it as a global value that components can access and subscribe to (we'll discuss this in our next topic).

Syntax:

```
1 const value = useContext(MyContext)
```

Example:

```
1 import { useContext } from 'react';
2
3 const ThemeContext = React.createContext('light');
4
5 const ThemedComponent = () => {
6   const theme = useContext(ThemeContext);
7
8   return <div className={`theme-${theme}`}>Current
9 };
10
11 export default ThemedComponent;
```

- This example sets up a **context** called **ThemeContext** with a default value of 'light'.
- Inside the **ThemedComponent**, the `useContext` hook accesses this context value to use the theme ('light').

useReducer

The **useReducer** hook is a more powerful alternative to **useState**.

It that allows you to manage state by state to a single value.

It takes a **reducer function** and **an initial state** as arguments, and returns the **current state** and a dispatch function.

The **reducer function** takes the **current state** and an **action** as arguments, and returns the next state. The dispatch function is used to dispatch actions to the reducer.

Example:



```
1 import React, { useReducer } from 'react';
2
3 const initialState = { count: 0 };
4
5 function reducer(state, action) {
6   switch (action.type) {
7     case 'increment':
8       return { count: state.count + 1 };
9     case 'decrement':
10      return { count: state.count - 1 };
11    default:
12      throw new Error();
13  }
14 }
15
16 function Counter() {
17   const [state, dispatch] = useReducer(reducer, initialState);
18
19   return (
20     <div>
21       <p>Count: {state.count}</p>
22       <button onClick={() => dispatch({ type: 'increment' })}>Increment</button>
23       <button onClick={() => dispatch({ type: 'decrement' })}>Decrement</button>
24     </div>
25   );
26 }
```

current state

new state

counter component

- The **reducer function** takes the **current state** and **an action** as arguments, and returns the new state based on the action type.
- The **dispatch function** is used to send actions to the reducer.

The benefits of using useReducer include:

- Easy to manage complex state logic
- Reducer function can be reused across multiple components.
- Dispatch function can be used to trigger state changes from anywhere in the component tree.

useRef

The useRef hook returns a **mutable ref object** that persists for the full lifetime of the component.

It is commonly used to access or manipulate DOM elements or **store mutable values**.

It takes an **initial value** as an argument and returns a **mutable object** with a **current property** that can be used to access the referenced value.

Syntax



```
1 const refContainer = useRef(initialValue)
```

useRef

The useRef hook returns a **mutable ref object** that persists for the full lifetime of the component.

It is commonly used to access or manipulate DOM elements or **store mutable values**.

It takes an **initial value** as an argument and returns a **mutable object** with a **current property** that can be used to access the referenced value.

Syntax



```
1 const refContainer = useRef(initialValue)
```

Example:



```
1 import React, { useRef } from 'react';
2
3 function FocusInput() {
4   const inputRef = useRef(null);
5
6   const handleFocus = () => {
7     inputRef.current.focus();
8   };
9
10  return (
11    <div>
12      <input ref={inputRef} type="text"
13      <button onClick={handleFocus}>Focus
14    </div>
15  );
16 }
```


useMemo

The **useMemo** hook **memoizes** a value by not recomputing it on every render.

It allows you to memoize (**cache**) the result of a function so that it's not recalculated.

It takes a **function** as an argument and returns a **cached** result.

Syntax

```
1 const memoizedValue = useMemo(() => computeExpensiveValue(a, b), [a, b])
```

- **computeExpensiveValue(a, b)** is a function that performs an expensive calculation

Example:



```
1 import React, { useState, useMemo } from 'react'
2
3 function ExpensiveCalculation({ num }) {
4   const calculate = (num) => {
5     console.log('Computing...');
6     return num * 2;
7   };
8
9   const result = useMemo(() => calculate(num), [num]);
10
11   return <div>Result: {result}</div>;
12 }
```

- The **ExpensiveCalculation** component takes a **num** prop.
- The **calculate function** performs the calculation.
- The **useMemo** hook is used to cache the result.

useCallback

Works like the `useMemo` hook, but instead of memoizing values, it memoizes functions to prevent it from being recreated on every render.

It is especially useful when passing callbacks to child components that rely on reference equality to prevent unnecessary re-renders.

Syntax



```
1 const memoizedFunction = useCallback(() => {  
  [dependencies];  
});
```

useCallback takes a function and **an array of dependencies**, and returns a **memoized function**.

Example:



```
1 import React, { useState, useCallback
2
3 function ParentComponent() {
4   const [count, setCount] = useState(
5
6   const increment = useCallback(() =>
7     setCount((c) => c + 1);
8   }, []); ← Empty dependency array
9             created once
10  return <ChildComponent increment={i
11 }
```

- The **ParentComponent** has a state initialized to 0.
- The increment function is defined using `useCallback`. It updates the count

Custom Hooks

A custom hook in React is a **reusable** function that uses other hooks to provide a specific functionality.

It's a way to extract component logic into a reusable function that can be used across components.

They follow the **naming convention** of **"use"** (e.g., useFetchData, useAuth). E



```
1 import { useState, useEffect } from 'react';
2
3 function useFetchData(url) {
4   const [data, setData] = useState(null); // State to hold fetched data
5   const [loading, setLoading] = useState(true); // State to track if
  data is being loaded
6
7   useEffect(() => {
8     async function fetchData() {
```

Breakdown of the Custom

- The **useState** initializes a state variable with an initial value of null and another variable **loading** with an initial value of false.
- The **useEffect** hook is used to perform side effects in function components. In this case, the side effect is fetching data from the provided URL.
- An **asynchronous function (fetchData)** is defined, which contains the logic for fetching the data, where **data** is the variable **response**.
- The **response** is then converted to a JSON format and stored in the variable **data**. This **data** is passed into the **setData** to update the state.

Using the custom hook (useFetchData) component



```
1 function App() {
2   const { data, loading } = useFetchData('https://api
  Using the custom hook
3
4   if (loading) {
5     return <div>Loading... </div>; // Show a loading message
    fetched
6   }
7
8   return (
9     <div>
10      <h1>Fetched Data:</h1>
11      <pre>{JSON.stringify(data, null, 2)}</pre> {/* I
    */}
12    </div>
13  );
14 }
```

- useFetchData is called with the URL ('https://api.example.com/data'). It

Conclusion

- **Class Components** are managed using lifecycle methods like `componentDidUpdate` to handle state effects. They can be more verbose and are being replaced by modern approaches.
- **Functional Components** leverage `useState` and `useEffect` to manage state and effects, making them more concise.

I hope you found this n
useful and helpful

Remember to:

Like

Save for future refer

Hi There!

Thank you for reading

Did you enjoy this knowledge



Follow my LinkedIn page for more
balancing and Coding tips



LinkedIn: Oluwakemi Oluwalan